



Course: AI and ML for Robotics

Lab Sheet 3: AIML Techniques for Sensor & Motion Modelling

Objective:

To implement AIML technique to replace the traditional Motion Model in robots. Below three parallel techniques are implemented on robots with varied dimensions of state/action space and comparative analysis is done to illustrate the challenge in learning the motion model.

Note: This is an incremental implementation. Hence the first few parts of the code base are the same as that of the Lab 1 .

Tools and Libraries Required:

Programming language: Python

Platform : Google Colab T4 GPU

Simulation : mujoco

Implementation pipeline

1. Data collection — datasets provide labelled data eg., One sample design could be $X = \text{concatenated state+action}$ and $Y = \text{state delta}$ used to fit dynamics models
2. Model training — Three parallel experiments: linear regression, MLP dynamics model, and DRL : PPO policy training.
3. Rollouts / evaluation — for dynamic models: open - loop rollouts where actions are sampled randomly and next state is predicted by model; for DRL-PPO: closed-loop policy rollouts.
4. Visualization — plotting the first state dimension over time to compare true vs predicted or policy trajectories.

Use case 1 : Machine Learning for Motion Modelling:

Implement Linear Regression to predict the next state(continuous) or delta change in the state values.

Design:

> Input : $X = (S, a)$

> Output : $Y_{\text{Pred}} = \Delta S$

> Training : Linear Regressor

> Loss Function : Mean Square Error where the predicted motion is compared against Y-actual environment states in simulator is observed for random actions & is used as target



Use Case 2 : Multi Layer Perceptron for Motion Modelling:

Implement MLP to predict the next state(continuous) or delta change in the state values.

Design:

- > Input : $X = (S, a)$
- > Output : $Y_{Pred} = \Delta S$
- > Training : MLP to perform regression with 2 hidden layer trained with ADAM optimizer
- > Loss Function : Mean Square Error where the predicted motion is compared against Y-actual environment states in simulator is observed for random actions & is used as target

Use Case 3 : Deep Reinforcement Learning for Motion Modelling:

Implement MLP to predict the next state(continuous) or delta change in the state values.

Design:

- > Input : Observations S_t from the environment(Simulation)
- > Output : Output action learnt by the policy
- > Training : PPO via stable baselines3 with MlpPolicy on a DummyVecEnv
- > Loss Function : Rollout_policy runs deterministic policy and plots state trajectories;
- >> Note: Proper evaluation should compute average episode return, success rate, and variance . This is not implemented in the uploaded lab sheet.

Practice Questions:

In Lab sheet 3(a):

1. In plots observed in the lab sheet code file, only the trajectory is shown. Students may add train/val/test splits and report numeric metrics (MSE, MAE, Average Rewards) rather than only plots
2. Instead of Delta change in state variable , if next state S_{t+1} needs to be predicted in ML as well MLP models, propose an architecture & implement the same
3. In DNN(MLP) section, if the environment can be modelled as partially observable or history dependent dynamics, try RNNs/GRUs/transformers that take past states/actions
4. In DRL section, try implementing tabular model based vs model free version of algorithm and comment your observation

In Lab Sheet 3(b)

1. In Linear Regression model , replace MSE with robust losses (Huber, Tukey) or use outlier detection before fitting
2. Replace synthetic dataset with sensor-specific data and validate on real sensor data.
3. Add adaptive calibration to track parameter drift over time.